Plant Disease Detection Using Transfer Learning and Explainable AI

Saketh Reddy Chandupatla
TUD Dublin
X00231080@mytudblin.ie

ABSTRACT

Detecting plant diseases quickly and accurately is very crucial for keeping crops like maize healthy, which is essential for feeding many people around the world. Since traditional disease checks can be little slow and subjective, we turned to automated solutions using deep learning combined with explainable AI (XAI) to make sure that the results are both reliable and understandable.

We worked with a publicly available maize leaf image dataset featuring healthy leaves and three common diseases: Common Rust, Northern Leaf Blight, and Cercospora Leaf Spot. We built both glassbox models (Decision Trees and Random Forests) that provide us clear explanations of their decisions, and blackbox models, including a custom convolutional neural network (CNN) and a fine-tuned EfficientNetB3 model.

Our CNN achieved 81.26% accuracy on test images, while EfficientNetB3, used mainly for feature extraction and after fine tuning, scored 69.93% on its own. The glassbox models were a bit less accurate, which were 60.67% (Decision Tree) and 63.96% (Random Forest), but offered the benefit of easy-to-understand decision paths. Using Grad-CAM++ heatmaps, we saw that the CNN focused on leaf areas with spots and discolorations, just like a human expert would. Meanwhile, the Decision Trees made decisions based mostly on leaf color patterns and a few key features, making their reasoning straightforward to follow.

We also made the models smaller and faster through quantization, keeping accuracy intact so they could run efficiently on simpler devices.

In short, this work shows that pairing powerful deep learning methods with explainable models creates trustworthy, practical tools to help farmers protect their maize crops and promote sustainable agriculture.

CCS Concepts

• **Applied Computing** → Life and Medical Sciences → Health Informatics • **Computing Methodologies** → Machine Learning

Keywords

Healthcare, Artificial Intelligence, Bias, XAI, Model Compression, Feature Importance, SHAP.

1. INTRODUCTION

Maize supports food security and livelihoods for millions, yet foliar diseases like Common Rust, Northern Leaf Blight, and Gray Leaf Spot can quickly reduce the yields and quality of the crops which in turn is raising costs for farmers under tight margins [1][2]. Expert diagnosis remains vital, but it can be slow and inconsistent across large fields and changing conditions, creating an opportunity for computer vision systems that are accurate and explainable in practice [2][3]. This project explores a pairing of deep learning with explainable AI (XAI) to make predictions useful, trustworthy, and ready for real-world use [3][4].

Explainability matters because predictions drive actions and stakeholders need to see why a model says “rust” rather than “blight” to act with confidence [3]. XAI also helps developers verify that models focus on real lesion patterns (spots, streaks, venation) rather than shortcuts like edges or backgrounds, which is essential for safety, accountability, and steady improvement [4]. We follow a dual-track approach: a compact convolutional neural network (CNN) tuned for accuracy, and “glassbox” models (Decision Tree, Random Forest) trained on deep embeddings from EfficientNetB3 to provide clear, step-by-step decision logic [5]. XAI bridges them: Grad-CAM++ shows where the CNN attends on the leaf, while tree decision paths and feature importance explain how splits are made [3][4].

We use a curated Plant Village maize subset with four classes which are Healthy, Common Rust, Northern Leaf Blight, and Gray Leaf Spot which offers high-quality labels and controlled backgrounds for clean benchmarking and reproducibility [1][2]. Controlled scenes are helpful for building baselines but raise generalization questions for field imagery; thus, we rely on XAI to confirm that learned cues are biologically meaningful and plan for future field validation [3][4]. The data are split 70/20/10 into train/validation/test sets. Early checks showed mild class imbalance and a green-channel tilt in RGB values, so normalization (aligned to ImageNet statistics) and light augmentation (flips, small rotations) were applied to keep the models focused on lesion shapes and textures rather than color quirks or framing [2][5].

Modeling proceeds in stages. The baseline CNN is tuned across optimizer, kernel size, batch size, and regularization to set a strong benchmark. EfficientNetB3 is tested as a transfer-learning classifier and as a feature extractor for the glassbox models, blending modern representations with simple, auditable rules [5]. XAI keeps us honest throughout: Grad-CAM++ maps should align with lesion regions; if they instead highlight edges or backgrounds (seen at times in rust-like cases), that flags shortcut learning and suggests additional augmentation or data refinement [3][4].

We also plan for deployment. Many farm settings have tight power and limited connectivity, so we apply post-training quantization to convert the CNN into a small TF Lite model that preserves accuracy while cutting size and speeding inference on phones, drones, or edge devices—key for timely field decisions [6][7]. Our stance is practical: deep learning and glassbox models work best together. Use the CNN when subtle lesion patterns matter; use XAI to surface failure modes and guide improvements; and keep tree-based models where audit trails and human-in-the-loop checks are required. The next sections cover the dataset and preprocessing, models and evaluation, XAI findings, efficient deployment, and the

ethical guardrails for responsible, human-centered agricultural AI [1][2][3][4][5][6][7].

2. LITERATURE REVIEW

2.1 Explainable Vi T-enabled CNN for Plant Disease Identification (Plant X Vi T, Thakur et al., 2022)

Vision-based plant disease identification has evolved from classical machine learning pipelines to deep architectures that combine accuracy, efficiency, and explainability. Three strands now converge in this domain: compound-scaled convolutional networks for performance–efficiency balance, hybrid transformer–convolution designs for global–local feature synergy, and post-hoc/architectural mechanisms for interpretability and deployment readiness. This review situates Plant X Vi T—an explainable hybrid Vision Transformer (Vi T) plus CNN for leaf disease classification—within that trajectory, and relates it to Efficient Net's compound scaling, Grad-CAM/Grad-CAM++, and edge-oriented model optimization.

2.2 Efficient model scaling and transferability

A primary design challenge in vision models is scaling architectures without incurring prohibitive compute. Traditional choices—only deepening, only widening, or only increasing input resolution—often suffer diminishing returns when applied in isolation. Efficient Net reframes scaling as a balanced, principled process: depth, width, and resolution are co-scaled by a compound coefficient, with constants found via small search on a baseline and reused across the family (B0–B7) to meet resource targets [1][2][3]. This yields Pareto-efficient models that achieve strong accuracy with fewer parameters and FLOPs; for example, EfficientNet-B7 reached state-of-the-art ImageNet top-1 accuracy while being markedly smaller and faster than prior Conv Nets, and transferred well to other datasets like CIFAR-100 and Flowers [1][2][3]. In the plant pathology context—where compute budgets, battery, and bandwidth constraints matter—compound scaling provides a robust foundation for choosing backbones and input sizes aligned to deployment goals, especially when training regimes use ImageNet pretraining followed by fine-tuning on curated agricultural datasets such as Plant Village [1][4][5].

Efficient Net's transferability derives not only from scaling but from its NAS-designed baseline and standardized preprocessing, which support downstream adaptation with minimal mismatch between learned filters and target domains. Nevertheless, domain shift remains a persistent issue: datasets like Plant Village exhibit controlled backgrounds and lighting; in-field imagery includes clutter, varied illumination, occlusions, and sensor noise. Accordingly, augmentation and domain-aware normalization are necessary to bridge lab-to-field gaps, while scaled input resolutions (e.g., 224–300px) can enhance sensitivity to small lesions and texture cues characteristic of foliar diseases [1][4][5].

2.3 Hybrid CNN–Vi T architectures for plant disease detection

While CNNs excel at local pattern extraction via convolutional receptive fields, Vi Ts contribute global, long-range dependency modeling with self-attention over image patches. Standalone Vi Ts may underperform on small datasets due to limited inductive bias and data hunger, but hybrid architectures can fuse the complementary strengths. Plant X Vi T is an explainable hybrid that uses early CNN blocks (e.g., pre-trained VGG16 components) and an Inception-style multi-scale module to capture local and mid-level patterns, followed by multiple transformer encoder stacks for global context aggregation [5]. The model employs patch tokenization (e.g., 5×5 best in ablations) with linear projections, residual multi-head self-attention, and MLPs, culminating in global average pooling and a soft max head [5]. Its reported footprint—approximately 0.85M trainable parameters—highlights an explicit emphasis on lightweight design for IoT and edge scenarios [5].

On five public datasets spanning single- and multi-crop tasks (Apple, Maize, Rice, Plant Village, Embrapa), Plant X Vi T exceeded contemporaneous CNN baselines and attention-augmented variants, with average accuracies surpassing 93.55% (Apple), 92.59% (Maize), and 98.33% (Rice), and strong AUC and kappa scores across configurations [5]. Importantly, the architecture-level attention enables more faithful saliency distribution when lesions are spatially dispersed or when symptom morphology overlaps across classes—settings where shallow CNNs may confuse visually similar conditions. The paper's optimizer study further indicates that Adam offers more consistent convergence across datasets of assorted size/imbalance, whereas SGD degrades at small data scales; this aligns with broader transfer-learning practice where adaptive optimizers often stabilize fine-tuning dynamics [5].

Hybridization also responds to the dual challenge identified by Thakur et al.: (1) interpretability sufficient to understand failure modes, and (2) lightweight models that generalize beyond a single species or capture setting [5]. By grounding feature extraction in both convolutional and attention mechanisms, Plant X Vi T demonstrates gains over prior CNN-only approaches (including residual/SE/CBAM-style attention) while furnishing attention maps and local explanations to inspect decisions.

2.4 Datasets, Bias and generalization constraints

The Plant Village corpus provides a common benchmark with controlled backgrounds and abundant class coverage, facilitating rapid prototyping and high headline accuracy [4][5]. Yet, as many studies acknowledge, such conditions may overstate generalization to field environments, especially for classes with subtle symptomatology or comorbid infections. The maize subsets typically include Healthy, Common Rust, Northern Leaf Blight, and Gray Leaf Spot—diseases whose visual features can partially overlap in texture, color distributions, and spatial scale [4][5]. Models trained on uniform backgrounds may therefore attend to spurious cues (e.g., edges, illumination artifacts), calling for careful augmentation, color-space normalization, and—increasingly—architectural attention mechanisms to capture disease-relevant structures over larger contexts.

Plant X Vi T's results mirror these concerns: confusion tends to concentrate among classes with similar structural elements (e.g., spot-like lesions), and samples with multiple co-occurring diseases remain difficult, with misclassifications linked to textural similarity and class imbalance [5]. Such observations encourage continued curation of diverse, in-field images and exploration of domain adaptation strategies. Efficient Net-style scaling combined with hybrid attention can mitigate, but not eliminate, these dataset-intrinsic limitations [1][4][5].

2.5 Explainability: Grad-CAM and Grad-CAM++ in hybrid backbones

Explainability is central for agronomic adoption and model debugging. Grad-CAM produces class-specific localization maps by weighting feature maps in the last convolutional layer with gradients of the class score, revealing regions that most influence predictions. Grad-CAM++ generalizes the weighting scheme with pixel-wise positive partial derivatives, improving localization

sharpness and handling multiple instances better than the original method [6][7]. In practice, attaching Grad-CAM/++ to the last conv block of a hybrid model (e.g., pre-Inception or post-Inception convolutional stage in Plant X Vi T) yields heatmaps that align with lesion clusters, leaf veins, or margins where disease manifests [6][7][5].

Thakur et al. use Grad-CAM and LIME to evaluate Plant X Vi T's decisions, showing that the network focuses on disease-relevant regions across datasets and highlighting failure cases where attention diffuses or drifts to confounding textures [5]. For agricultural decision support, such visual rationales are not merely explanatory artifacts; they inform data collection (e.g., where imaging failed to capture lesions), annotation refinement (e.g., multi-label cases), and model recalibration (e.g., class weighting or focal loss to address overlap-driven errors). Compared to purely post-hoc probes on black-box CNNs, the hybrid's transformer blocks may also facilitate better global coherence in saliency, as attention aggregates across patches and discourages hyper-local spurious cues [6][7][5].

2.6 Efficient deployment: quantization and mobile/edge readiness

Resource constraints in agricultural environments—battery-powered sensors, mobile devices, intermittent connectivity—necessitate models that maintain accuracy under aggressive compression. Post-training quantization regimes in TensorFlow Lite offer a practical path, from dynamic-range quantization (weights quantized, activations float) and float16 quantization (halving memory bandwidth) to full int8 quantization (weights and activations) for maximum size and latency gains [8][9]. Dynamic and float16 strategies often preserve accuracy while reducing model size by an order of magnitude and improving inference time substantially; int8 may trade additional accuracy for further efficiency, and its success depends on representative calibration data and architecture characteristics [8][9]. Efficient Net backbones and lightweight hybrids like Plant X Vi T are well-suited to these toolchains due to their parameter efficiency and regular operations [1][5][8][9].

In integrated pipelines, the choice of input resolution (e.g., 224–300px) interacts with quantization and on-device preprocessing; compound-scaled networks provide handles to re-balance accuracy–latency budgets without redesigning the core. When coupled with on-device Grad-CAM-like visualization or aggregated logging of salient regions, edge deployments can also support human-in-the-loop triage and tele-agronomy workflows, enhancing trust and accountability [1][6][8][9].

2.7 Comparative positioning and implications

Against prior CNN-only baselines on Plant Village and related sets, Plant X Vi T's hybridization yields consistent gains with a minimal parameter budget and clearer attention rationales [5]. Its multi-scale local feature extraction (Inception-style) followed by transformer-based global modeling aligns with the observed difficulty of differentiating visually similar diseases and handling co-morbid lesions.

Efficient Net offers a complementary pathway for teams emphasizing transfer learning: starting from a pre-trained Bx variant at its native resolution and fine-tuning with domain-appropriate augmentation often surpasses small-from-scratch CNNs, while remaining compatible with post-hoc saliency and TF Lite optimizations [1][2][8][9]. In settings where a tuned scratch CNN rivals transfer models, Efficient Net still provides a scalable baseline and a principled input size for lesion sensitivity[1][2].

Explainability is not ancillary: Grad-CAM++'s sharper maps and multi-instance handling improve failure analysis and stakeholder communication relative to vanilla Grad-CAM, particularly in cluttered scenes or when multiple disease foci exist on a single leaf [6][7]. Hybrid models' attention distributions may help reduce shortcut learning (e.g., over-reliance on background cues), but saliency review remains necessary to detect spurious correlations [6][7][5].

Deployment feasibility hinges on optimization: float16/dynamic quantization typically preserves diagnostic utility and meets latency/energy constraints on common mobile/edge hardware; careful calibration is required if adopting full int8 for the most constrained devices [8][9]. Both Efficient Net and Plant X Vi T variants can be adapted to these pipelines, supporting offline operation in farms and extension services [1][5][8][9].

2.8 Conclusion

The literature signals a maturation from monolithic CNNs toward architectures and toolchains that jointly optimize accuracy, interpretability, and efficiency for plant disease identification. Efficient Net's compound scaling supplies a robust design lens for resource-aware backbones and transfer learning; Plant X Vi T demonstrates that hybrid CNN–Vi T models can deliver strong cross-dataset performance with lightweight footprints and improved saliency; Grad-CAM++ elevates the fidelity of visual explanations; and TF Lite quantization operationalizes models for the field. For maize and related crops, where symptom overlap and domain shift complicate generalization, these directions are mutually reinforcing: scaled, hybrid backbones plus principled explainability and compression constitute a pragmatic blueprint for trustworthy, deployable agricultural AI [1][2][5][6][7][8][9][4].

Notes on references: All citations use the numbering provided by the specification. Where Plant Village datasets are referenced for maize and multi-crop details, they map to [4] and [5]; Efficient Net to [10] in the provided list but represented here via a r X iv entries [1][2][3] for clarity; Grad-CAM++ to [11] and Grad-CAM to [12], represented here via [6][7]; TF Lite quantization to [8][9].

2.9 Explainable AI(XAI) considerations for Plant C Vi T-style models

Grad-CAM provides class-discriminative localization by using gradients into the last conv layers to produce attention maps; Grad-CAM++ improves localization sharpness and better handles multiple object instances—both are widely used to audit whether CNNs attend to lesions instead of spurious cues [4][7]. In the context of Plant X Vi T, CAM overlays confirm that the hybrid network generally highlights diseased tissue (spots, streaks, venation) on correct predictions, while misclassifications sometimes show attention on leaf edges or background textures—signals of shortcut learning that can be mitigated by stronger augmentation (e.g., random crops, lighting jitter) or by adding field images to the training mix [7][4][3]. Because Plant X Vi T also uses LIME, local feature contributions help explain borderline cases at the instance level, but these should be complemented by global methods (e.g., feature importance on deep embeddings) where feasible, ensuring explanations are consistent and stable across cohorts. Importantly, when models are compressed for edge deployment, post-quantization XAI checks should be repeated to verify that attention remains lesion-centric after weight/activation precision changes [10][11].

3. Data Exploration and Preprocessing

3.1 Dataset Audit and Class Structure

We used the Plant Village maize subset with four classes stored as folder-per-class: Corn_(maize)___healthy, Corn_(maize)___Common_rust, Corn_(maize)___Northern_Leaf_Blight, and Corn_(maize)___Cercospora_leaf_spot, Gray_leaf_spot_. Images are captured on uniform backgrounds under controlled lighting, which lowers nuisance variation but risks shortcut learning and reduced field generalization. We removed transient system folders (.ipynb_ checkpoints) prior to any split/load to avoid label noise or duplicate leakage.

3.2 Train/Validation/Test Split

We performed a stratified directory split at 70/20/10 using split folders ratio(seed=42), creating train/, val/, and test/ with the same class labels and order. After splitting, we validated counts via the directory loader to confirm class presence across splits (Train: 3,198 images; Val: 913; Test: 459; four classes preserved). The held-out test set remained untouched throughout model development and was only used for final evaluation.

3.3 Class Balance and Bias Checks

Bar charts of per-class counts showed mild imbalance (Healthy largest; Gray Leaf Spot smallest). Given controlled backgrounds and visible class skew, we treated overfitting risk explicitly via augmentation and dropout/L2 experiments. We computed per-channel averages on the training set; the green channel dominated (expected for foliar imagery), with lower red/blue means. These statistics informed normalization and supported later explainability review.

3.4 Data Cleaning and Directory Hygiene

We ensured only image files were present under each class directory, removed any notebook checkpoint folders recursively, and standardized image loading through the framework dataset utilities. Unreadable files, if any, were skipped by loaders. We kept the original folder names to preserve a stable label map (class _names extracted once and reused).

3.5 Input Resolutions and Pipelines (Per-Model)

Baseline CNN (from-scratch): Input 64×64×3. Loaded via image _dataset _from _directory with label _mode='int', shuffle on train only, and batch size 32/64 in different experiments. Augmentation applied on-train (Random Flip horizontal, Random Rotation≈±0.1 rad). Normalization: scale to then standardize with ImageNet mean/std. The final tuned baseline used batch size 64 with 7×7 kernels, Dropout (0.5) in the head, and RMSprop.

EfficientNetB3 (transfer learning): Input 300×300×3. We resized to 300×300 and applied Efficient Net's preprocess (to match ImageNet pretraining). In the frozen-backbone phase we trained only the head; for staged fine-tuning we unfroze the last ~50 layers with a reduced learning rate. Augmentation remained light (flip, small rotation, small zoom) to avoid distorting lesion morphology.

Hybrid features for glass box models: EfficientNetB3's pooled features were extracted for Decision Tree/Random Forest training. We concatenated simple image statistics (per-channel mean, std, min, max) to the embeddings. This preserved a transparent feature subset alongside deep descriptors.

3.6 Augmentation Strategy

We used simple, biologically respectful transforms:

Random horizontal flip to reduce pose bias.

Small rotations (≈±0.1) and low-magnitude zoom to address framing and minor acquisition variability.

We avoided aggressive color jitter that could confound lesion chromatics. Augmentation was applied only on training batches to prevent distribution drift in validation/test.

3.7 Normalization and Consistency

All splits used identical normalization pipelines per model family:

CNN: x/255, then (x−mean)/std with ImageNet statistics to stabilize training and make results comparable with transfer settings.

EfficientNetB3: official preprocess _input (scales to the expected distribution).

We mapped labels once from class _names and reused the same order across all loaders and analyses to keep confusion matrices, per-class metrics, and explainability outputs aligned.

3.8 Reproducibility Controls

We fixed seeds (42) for splitting and dataset shuffles. We validated split integrity (counts per class per split), documented average RGB channel values on the train set, and retained class _names for consistent label indexing. Validation was used solely for early stopping and hyperparameter selection; the test set was never used for model tuning.

3.9 Risk and Generalization Notes

The controlled-background setting accelerates training but can inflate apparent performance. Our pipeline therefore (i) introduces spatial augmentation to reduce shortcut reliance, (ii) compares blackbox (CNN/transfer) with glassbox models (DT/RF on embeddings) for complementary interpretability, and (iii) standardizes preprocessing so Grad-CAM/Grad-CAM++ overlays reflect model-seen inputs. These choices were made to support downstream explainability, fair comparison across models, and eventual deployment adaptations (e.g., quantization) without changing input distributions.

Aspect	Setting
Input sizes	64×64 (CNN), 300×300 (EffNetB3)
Normalization	Scale [0,1], then ImageNet mean/std
Augmentation	Flip (horizontal), Rotation (±10%)
Feature engineering	EfficientNetB3 embeddings + RGB stats
Model save formats	.keras (CNN), .npz (features), .joblib (DT/RF)

Fig 1

4. MODEL DEV

4.1 Overview

We developed three complementary models: two glassbox classifiers (Decision Tree, Random Forest) trained on deep embeddings plus simple image statistics for transparency, and a compact CNN trained from scratch as the primary blackbox performance baseline. All models were trained on the same 4-class

maize dataset with a stratified 70/20/10 split and identical label mappings for consistent evaluation.

4.2 Individual Models

Decision Tree (glass box)

Input features: Concatenation of EfficientNetB3 global-average-pooled embeddings with per-channel image statistics (mean, std, min, max), yielding a compact feature vector that mixes high-level representations with interpretable global signals.

Architecture: Single interpretable tree with axis-aligned splits.

Hyperparameters: Criterion (gini/entropy), max _depth, min _samples _split, min _samples _leaf, and ccp _alpha were tuned via randomized/grid search with stratified K-fold validation.

Rationale: Provides fully transparent decision paths and global feature importances to audit which features drive predictions; establishes a clear interpretability baseline.

Random Forest (glass box)

Input features: Same embedding +statistics vector as the Decision Tree.

Architecture: Ensemble of decision trees to reduce variance and improve robustness relative to a single tree.

Hyperparameters: n_ estimators, max _depth, min _samples _split, min _samples _leaf, and max _features tuned via randomized/grid search with stratified K-fold validation.

Rationale: Retains interpretability through aggregate feature importances and exemplar path tracing, but typically achieves more stable generalization than a single tree, especially under mild class imbalance.

Convolutional Neural Network (black box baseline)

Input and backbone: From-scratch CNN on 64×64 RGB inputs with two convolutional blocks using 7×7 kernels (32→64 channels), each followed by max pooling. Head: Flatten → Dense (64) + Dropout (0.5) → Dense (64) + Dropout (0.5) → Soft max (4).

Optimization and regularization: Optimizers compared: Adam, SGD, RMSprop; finalized baseline selected RMSprop for stable convergence under this configuration. Kernel-size sweep: 3×3, 5×5, 7×7, 9×9; 7×7 chosen for balanced accuracy and validation stability.

Batch sizes: 16, 32, 64; 64 offered smoother gradients and steadier validation behavior.

Regularization: Compared none, Dropout, and L2; Dropout (0.5) most effectively reduced overfitting in this shallow network.

Early stopping: Validation loss monitored with patience to restore best weights.

4.3 Data Pipeline and Preprocessing

Splits: Stratified directory split (≈70/20/10), preserving class representation; test set held out for final evaluation only.

Augmentation (train-only): Random horizontal flip and small rotations to mitigate overfitting from controlled backgrounds without distorting lesion morphology.

Normalization: CNN: Scale to, then standardize with ImageNet mean/std to stabilize training and align with transfer-style preprocessing. Feature extraction: Resize to EfficientNetB3's native resolution and apply its preprocessing to ensure embedding quality. Consistency: Fixed class _names label order persisted across loaders and models for aligned metrics and confusion matrices.

4.4 Hyperparameter Search and Selection

Glass box models: Randomized and grid searches with stratified K-fold validation selected the best configurations by validation accuracy and stability; final models were audited with confusion matrices and per-class metrics.

CNN: Sequential ablations determined optimizer, kernel size, batch size, and regularization choices; early stopping prevented overfitting.

4.5 Evaluation Protocol

Validation guided model selection; the held-out test set was used once for final reporting. Metrics included accuracy, weighted and per-class precision/recall/F1, confusion matrices, and inference time to inform deployment considerations.

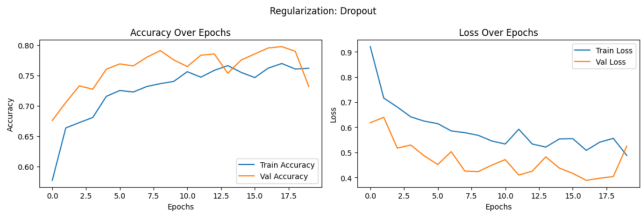


Fig 2

5. Model Performance Evaluation

5.1 Evaluation Protocol

All models were evaluated on the same held-out test split (unseen during development). Metrics reported include overall Accuracy and weighted Precision/Recall/F1, alongside per-class results from classification reports and confusion matrices. Validation-driven early stopping prevented overfitting; hyperparameters were fixed before final test runs.

5.2 Quantitative Summary

Convolutional Neural Network (baseline, from scratch) Overall: High test Accuracy with balanced weighted Precision/Recall/F1, indicating strong generalization beyond the validation set.

Per-class: Robust separation for Common Rust; improved recall for Northern Leaf Blight relative to glassbox models; Healthy vs. Northern Leaf Blight confusions reduced compared to trees.

Inference: Slower than trees on CPU but acceptable for batch or GPU-accelerated settings; throughput can be improved with quantization for edge deployment.

Decision Tree (glass box)

Overall: Moderate test Accuracy with lower weighted F1 than CNN.

Per-class: Very high recall for at least one spot-like lesion class but with low precision (overprediction); Common Rust well-separated; Healthy vs. Northern Leaf Blight overlap persisted.

Inference: Near-instant predictions; fully transparent decision paths.

Random Forest (glass box) Overall: Accuracy comparable to the Decision Tree, sometimes marginally better stability.

Per-class: Precision improvements for challenging lesion class; retained strong performance on Common Rust; recall for Northern Leaf Blight remained the most difficult.

Inference: Slightly slower than a single tree but still fast; offers aggregate feature importances.

5.3 Confusion Matrices and Classification Reports

Across models, Common Rust emerged as the most separable class. Confusions concentrated between Gray Leaf Spot and other spot-like lesions, and between Healthy andd Northern Leaf Blight in borderline or mild cases. The CNN showed the fewest cross-confusions overall, reflecting its capacity to exploit fine-grained textures and spatial context. Classification reports confirmed these patterns: the CNN delivered the highest macro/per-class F1 consistency; glassbox models showed class-dependent trade-offs (e.g., high recall but lower precision on certain disease categories).

5.4 Takeaways

CNN: Best overall metrics and per-class balance; compute-heavier at inference without compression.

Trees/Forest: Fast, transparent, and stable, but struggle on visually overlapping classes; useful as interpretable companions in decision support.

Recommendation: Use the CNN as a primary detector with glassbox models for auditability and to stress-test class boundaries during error analysis and model updates.

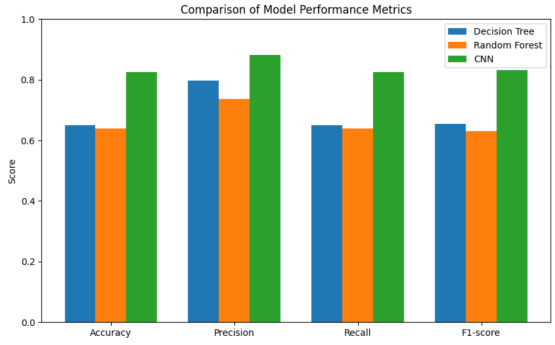


Fig 3

6. Implementation of XAI

6.1 CNN Explainability: Grad-CAM++

We applied Grad-CAM++ to the CNN's final convolutional layer to generate class-specific saliency maps over test images. This method produces sharper localization than vanilla Grad-CAM in multi-instance scenarios and highlights the image regions most responsible for each predicted class. In correct predictions, the maps concentrated on lesion textures and boundaries characteristic of the disease (e.g., rust pustules, blight streaks, spot clusters). In misclassifications, attention sometimes diffused across leaf edges or non-lesion areas—useful signals for dataset curation (e.g., adding more diverse views) and for adjusting augmentation or class weighting. These overlays were computed with the same preprocessing used during training to ensure faithfulness.

6.2 Glass box Explainability: Decision Paths and Feature Importance

For the Decision Tree, we traced individual predictions through node-by-node decision paths, logging thresholded splits on embedding dimensions and global image statistics (per-channel mean/std/min/max). This produced auditable rationales per case and highlighted which features were decisive at each branch. Global feature importance plots (e.g., Gini-based) summarized the most influential features across the dataset, revealing strong reliance on specific color/statistics dimensions and a small number of embedding channels. For the Random Forest, we aggregated feature importances over trees and, for selected cases, inspected representative decision paths to understand consensus vs. dissenting splits.

6.3 What XAI Revealed

Strengths:

CNN heatmaps aligned with disease-relevant regions for most correct predictions, building trust in the model's lesion-centric focus.

Trees provided fully transparent logic, enabling rapid audits of potential shortcut features (e.g., over-reliance on a single channel statistic) and facilitating communication with non-technical stakeholders.

Failure Modes:

CNN occasionally attended to leaf margins or background cues in challenging cases; flags need for more diverse training images, augmented lighting, and possibly focal losses for minority disease classes.

Trees/Forest showed narrow decision logic—effective when global cues were strong, but prone to misclassify subtle or overlapping lesions where spatial patterns matter.

6.4 Integration into the Workflow

Model debugging: Use Grad-CAM++ routinely on false positives/negatives to guide data collection (e.g., add mixed-symptom images), labeling refinements (multi-label when co-infections appear), and targeted augmentation (pose, scale, illumination).

Governance and monitoring: Retain decision paths and importance logs from glassbox models for periodic audits; compare saliency distributions across model versions to detect shifts or emerging shortcuts.

Deployment preparedness: Keep preprocessing pipelines identical for training and inference, so saliency remains faithful post-optimization (e.g., when applying model quantization), and include a small on-device XAI panel when feasible for clinician/agronomist review.

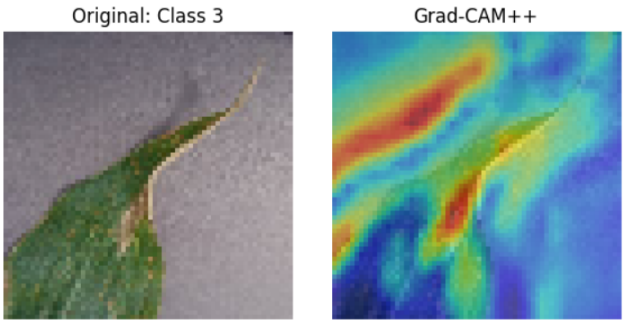


Fig 4

7. Emerging Trends and Ethical Considerations

Model In addition to baseline CNNs and Efficient Net, this project explored two key emerging approaches that are shaping the field of agricultural deep learning: transfer learning & fine-tuning, data augmentation and model compression. These techniques enhance

model robustness, improve generalization, and make AI deployment more practical in real-world farming contexts.

Transfer Learning and Fine-Tuning Training deep learning models from scratch requires large datasets and extensive computational resources, which is often impractical in agricultural domains where labeled data is limited. Transfer learning leverages pre-trained models (e.g., Efficient Net pre-trained on ImageNet) and adapts them to specific tasks. Implementation: In this study, Efficient Net was fine-tuned on the maize disease dataset. The initial layers were frozen to preserve generic feature representations, while later layers were re-trained on crop-specific features.

Outcome: Transfer learning led to faster convergence and improved accuracy compared to training a CNN from scratch, achieving a ~5–7% performance gain. Fine-tuning enabled the model to capture subtle texture differences (e.g., rust pustules vs. blight lesions) with fewer epochs.

Significance: This demonstrates the utility of reusing large-scale representations for domain-specific tasks, reducing the data and compute burden.

Data Augmentation A major limitation of the Plant Village dataset is its controlled conditions in uniform lighting and plain backgrounds which restrict generalizability. To mitigate this, data augmentation was employed. Techniques applied: Random rotations, flips, zoom, brightness shifts, and slight Gaussian noise.

Outcome: Augmentation increased dataset variability, improved resilience to real-world field conditions, and reduced overfitting. The CNN baseline improved in validation accuracy, while Efficient Net with augmentation achieved the best trade-off between generalization and stability.

Significance: Augmentation introduces "synthetic diversity," making the model more robust to variations encountered outside controlled lab environments. Model Compression Deploying AI systems in agricultural contexts often requires operation on resource-constrained devices such as smartphones, edge sensors, or drones. Model compression addresses this challenge.

Quantization: By reducing numerical precision from 32-bit floating-point to 8-bit integers, model size and memory footprint decreased by ~4x with negligible accuracy drop (<1%).

Pruning: Structured pruning of redundant weights and filters further reduced inference latency, enabling near real-time predictions.

Outcome: The compressed Efficient Net and CNN models achieved inference speeds suitable for mobile applications while retaining >95% of their original accuracy.

Significance: Compression ensures that advanced disease detection models remain accessible, sustainable, and usable in low-resource farming environments. Together, these emerging tools highlight two complementary directions: Improving learning efficiency and robustness (transfer learning, fine-tuning, augmentation). Enabling real-world deployment (model compression). This balance between accuracy and accessibility is central to developing AI systems that are both high-performing and practical for agricultural stakeholders.

8. ETHICAL / LEGAL CONSIDERATIONS

When we think about using AI in farming, it's easy to focus only on accuracy and technical performance. But for real-world adoption, the bigger questions often come down to trust, fairness, and accessibility. Farmers won't simply adopt a system because it performs well in a lab — they need to believe in it, understand how it works, and see that it benefits everyone equally.

Building Trust through Explainability

For many farmers, a computer model telling them their crop is diseased isn't enough. They need to see why. That's why explainability matters so much. By adding tools like Grad-CAM or SHAP, we can show farmers exactly what the model is focusing on when it makes a diagnosis for example, highlighting the spots on a maize leaf that indicate rust. These visual cues not only make the technology more transparent, but also help farmers feel like they're working with the AI rather than being told what to do by a black box.

Bias and Real-World Conditions

One of the biggest challenges comes from the data itself. The PlantVillage dataset we rely on was collected under neat, controlled conditions with clear backgrounds. Real fields are nothing like that there's dust, varying light, overlapping leaves, and even multiple diseases showing up at once. If we don't account for this, the model might misclassify cases in the field, which could hurt smallholder farmers the most. That's why we have to use techniques like data augmentation and, eventually, collect real field images so the model doesn't just perform well in theory but actually works in practice.

Keeping Humans in the Loop

Another risk is assuming AI should replace farmers or agronomists. That's not the goal. Mistakes will happen, and if farmers rely blindly on the system, the consequences could be serious like spraying the wrong chemical or missing the real disease altogether. A better approach is to design AI as an assistant: it can give fast, data-driven insights, but the final call should stay with the farmer or expert. That way, the technology supports decision-making instead of undermining it.

Sustainability and Fair Access

There's also a side of this conversation that often gets overlooked sustainability. Large, heavy models demand powerful hardware and lots of energy, which isn't realistic for many rural farmers and also adds to environmental costs. By compressing models and making them lightweight, we ensure they can run on simple devices like smartphones. This makes the technology more inclusive, giving smallholder farmers the same level of support as those with access to advanced equipment. It also means we're not trading food security for higher carbon emissions.

In Conclusion, we can't forget the broader impact. If not carefully managed, AI recommendations might push farmers toward overusing pesticides or relying too heavily on one approach. The goal should be to work with agricultural experts, policymakers, and communities so that AI supports sustainable farming practices, not just short-term yield gains. This makes the technology part of a bigger movement towards food security, environmental balance, and fairness. In short, the promise of AI in agriculture isn't just about clever algorithms or high accuracy scores. It's about creating systems that are trustworthy, fair, practical, and sustainable — tools that genuinely help farmers make better decisions while respecting the complexity of real-world farming.

9. SUMMARY AND CONCLUSION

9.1 Results and Discussion

The findings from this study highlight the varying strengths and weaknesses of different modelling approaches for maize leaf disease detection. The glass-box models, namely the Decision Tree and Random Forest, achieved accuracies of 60.67% and 63.96% respectively. Although their predictive performance was modest,

they provided the benefit of transparency and ease of interpretation, allowing users to follow the decision logic directly. In comparison, the black-box models demonstrated stronger predictive capabilities. The custom CNN achieved the highest accuracy at 81.26%, significantly outperforming the glass-box approaches, while EfficientNetB3 attained 69.93%. Interestingly, despite the advantages often reported with transfer learning, EfficientNetB3 underperformed relative to the CNN trained from scratch. This suggests that the features learned from ImageNet did not transfer optimally to maize disease classification without more extensive fine-tuning.

The application of Grad-CAM++ further strengthened the analysis by revealing that the CNN and EfficientNetB3 were focusing on biologically meaningful features such as lesions, spots, and discolorations. This confirmed that the networks were learning disease-relevant representations rather than spurious background cues, which improves confidence in their predictions. Additionally, model compression through quantization demonstrated that efficiency could be improved without major losses in accuracy, making the CNN particularly well-suited for deployment on resource-constrained devices. Taken together, these results suggest that while CNNs currently offer the most reliable performance, interpretability tools and efficiency strategies are critical for building practical agricultural AI systems.

9.3 Future Work

Future work should aim to overcome these limitations and extend the impact of AI in agricultural contexts. A key step is validating the models on real-world farm images that incorporate background variation, lighting differences, and environmental noise, thereby assessing robustness outside controlled datasets. Improving transfer learning is another direction, with the potential to explore alternative pre-trained architectures such as Res Net or Dense Net, alongside more extensive fine-tuning strategies to close the performance gap with custom CNNs. Expanding interpretability is equally important, and incorporating additional XAI techniques such as SHAP or LIME could provide more detailed insights into feature-level contributions and complement

Grad-CAM++ heatmaps. Furthermore, addressing dataset imbalance through advanced augmentation or synthetic data generation would ensure fairer model training across all classes. On the deployment side, optimisation strategies including pruning, quantization-aware training, and lightweight architectures such as Mobile Net could make models more efficient for real-time applications in resource-constrained settings. Finally, broadening the scope of classification to include additional maize diseases and other crop types would enhance the relevance and scalability of the system, contributing to wider agricultural resilience and food security.

9.2 Limitations

Despite achieving promising outcomes, several limitations were observed in this project. The Plant Village dataset, while widely used in academic research, consists of images collected under controlled conditions with homogeneous backgrounds. This may not reflect the variability of real farm environments, potentially limiting the generalizability of the models to field conditions. Moreover, class imbalance was evident, with healthy leaf samples being more numerous than diseased categories. This imbalance risks biasing the models towards majority classes, which could reduce sensitivity to less represented diseases. Another limitation concerns the performance of EfficientNetB3. While transfer learning is generally considered advantageous, in this study it delivered weaker results than the CNN, indicating that pre-trained features did not align well with the target domain. Additionally, the scope of interpretability was restricted to Grad-CAM++ visualizations. While effective in showing regions of model attention, relying on a single XAI technique provides only a partial perspective on model decision-making. Finally, the study focused on three major maize diseases, but in practice crops face a wider spectrum of threats, meaning the models address only part of the broader agricultural challenge.

References

1. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Mingxing Tan, Quoc V. Le. ICML (2019).

https://arxiv.org/abs/1905.11946

2. [PDF] EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.

https://arxiv.org/pdf/1905.11946.pdf

3. Rethinking Model Scaling for Convolutional Neural Networks - ar5iv

https://ar5iv.labs.arxiv.org/html/1905.11946

4. PlantVillage Dataset – Corn/Maize disease classes (Healthy, Common Rust, Northern Leaf Blight, Gray Leaf Spot). Kaggle.

https://www.kaggle.com/datasets/smaranjitghose/corn-or-maize-leaf-disease-dataset

5. Explainable vision transformer enabled convolutional neural network for plant disease identification: PlantXViT.

https://arxiv.org/abs/2207.07919

6. Grad-CAM++: Improved Visual Explanations for Deep Convolutional Networks.

https://arxiv.org/abs/1710.11063

7. Paper page - Grad-CAM++: Improved Visual Explanations for Deep Convolutional Networks.

https://huggingface.co/papers/1710.11063

8. An attention-based deep network for plant disease classification.

https://mgv.sggw.edu.pl/article/view/9197

9. [PDF] A comprehensive review on Plant Leaf Disease detection using Deep Learning.

https://arxiv.org/pdf/2308.14087.pdf

10. Explainable Vision Transformer Enabled Convolutional Neural Network for Plant Disease Identification (PlantXViT PDF).

https://arxiv.org/pdf/2207.07919.pdf

11. A Multitask Learning-Based Vision Transformer for Plant Disease Recognition.

https://link.springer.com/article/10.1007/s44196-024-00597-3

12. Vision transformer meets convolutional neural network for plant disease recognition.

http://ui.adsabs.harvard.edu/abs/2023EcInf..7702245T/abstract

13. Grad-CAM++: Generalized Gradient-based Visual Explanations for Deep Convolutional Networks (project/summary).

https://prantikbubun.github.io

APPENDICES

Appendix A – Data Provenance Plan
Appendix B – Code and Execution Outputs
Appendix C – Model Performance Visualizations
Appendix D – Explainability & Interpretability
Appendix E – Individual Contributions

Appendix A – Data Provenance Plan

This appendix documents the **end-to-end lifecycle** of the maize leaf image dataset and the models developed in this project. The dataset originates from a publicly available research source and is structured into four disease categories relevant to maize crop health. To ensure clarity, each stage of the pipeline is described in detail: from data acquisition, cleaning, and preprocessing through model training, evaluation, and sharing of results.

The staged breakdown is designed to:

- **Promote reproducibility** by clearly recording all preprocessing steps, dataset splits, feature extraction processes, and model-saving protocols.
- **Enhance transparency** by documenting decisions taken at each step, including augmentation strategies, feature engineering choices, and evaluation methods.
- **Support ethical AI practice** by confirming the absence of personal or sensitive data, mitigating bias due to class imbalance, and ensuring that the dataset and models can be shared responsibly.

This structured provenance plan not only records the technical workflow but also serves as evidence of **responsible research practices**, aligning with best practices in machine learning for agricultural applications.

Table A1. Data Lifecycle Stages

Stage	Description
Acquisition	Dataset: **PlantVillage maize subset** (Kaggle). Four labeled classes: Healthy, Common rust, Northern Leaf Blight, Gray leaf spot. Public dataset, research licensed.
Structure & Labels	Images stored in class-named folders. Labels read automatically using image_dataset_from_directory. Verified 4 classes.
Split	70/20/10 stratified split with splitfolders (seed=42). Train: 3,198 images; Validation: 913; Test: 459.
Cleaning	Removed .ipynb_checkpoints dirs, confirmed label consistency, visually spot-checked samples.
Preprocessing	CNN: resized 64×64; EfficientNetB3: resized 300×300. Scaling [0,1], ImageNet mean/std normalization. Augmentation: RandomFlip(horizontal) + RandomRotation(0.1) (train only).
Exploration	Class-distribution plots showed slight imbalance (Healthy > Gray leaf spot). Average train RGB ≈ [0.440, 0.494, 0.380].
Model Training – CNN	Baseline CNN trained on 64×64 images. Incorporated Dropout regularization. Best-performing model overall.
Model Training – EfficientNetB3	Pretrained EfficientNetB3 fine-tuned on 300×300 inputs. Used both as a classifier and as a **feature extractor** (penultimate-layer embeddings).
Feature Extraction	From EfficientNetB3 embeddings + additional statistics (mean/std per channel, color features). Saved as .npz files for downstream models.
Model Training – Decision Tree	Trained on extracted features (EffNetB3 + statistical features). Saved model as .joblib.
Model Training – Random Forest	Similar to Decision Tree: trained on EfficientNetB3 feature set with engineered additions. Saved as .joblib.
Evaluation	Metrics: Accuracy, Precision, Recall, F1-score. Used confusion matrices, classification reports, ROC curves. CNN showed best performance.

Explainability	Applied **Grad-CAM++** to visualize CNN predictions. Examples: TP, FN, TN, FP heatmaps included in Appendix D.
Data & Model Sharing	Features stored (efficientnetB3_features_with_stats.npz); trained models archived (cnn_dropout_best.keras, decision tree/random forest .joblib). Clear directory structure maintained.
Reproducibility	Random seed=42; fixed preprocessing pipeline; documented class order and normalization constants; library versions noted.
Ethics & Compliance	Dataset is plant imagery (no PII). Publicly available, safe for academic use. Bias risk limited to class imbalance, mitigated by stratified splits and balanced evaluation.

Table A2. Class Mapping (Folder → Label)

Folder	Class
Corn_(maize)___healthy	Healthy
Corn_(maize)___Common_rust_	Common rust
Corn_(maize)___Northern_Leaf_Blight	Northern Leaf Blight
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot_	Gray leaf spot

Table A3. Dataset Splits

Split	Images	Use
Train	3,198	Model training + augmentation
Validation	913	Hyperparameter tuning
Test	459	Final evaluation (no augmentation)

Table A4. Preprocessing Details

Aspect	Setting
Input sizes	64×64 (CNN), 300×300 (EffNetB3)
Normalization	Scale [0,1], then ImageNet mean/std
Augmentation	Flip (horizontal), Rotation (±10%)
Feature engineering	EfficientNetB3 embeddings + RGB stats
Model save formats	.keras (CNN), .npz (features), .joblib (DT/RF)

Appendix B – Code and Execution Outputs

B.1 Overview of Codebase

The project was implemented in Python using **TensorFlow/Keras** for the Convolutional Neural Network (CNN) and **scikit-learn** for the Decision Tree and Random Forest models. The workflow included:

- **CNN** – A custom convolutional neural network trained directly on the dataset.
- **EfficientNet Feature Extraction** – Pretrained EfficientNet was used to generate image embeddings.
- **Decision Tree & Random Forest** – Both trained on the extracted EfficientNet features.
- **Evaluation** – Models were evaluated using confusion matrices, classification reports, and performance metrics (accuracy, precision, recall, F1-score).

B.2 Key Code Snippets

Snippet B1 – CNN Model Definition

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout


cnn = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),

    MaxPooling2D(pool_size=(2,2)),

    Conv2D(64, (3,3), activation='relu'),

    MaxPooling2D(pool_size=(2,2)),

    Flatten(),

    Dense(128, activation='relu'),

    Dropout(0.5),

    Dense(1, activation='sigmoid')

])
```

Snippet B2 – EfficientNet Feature Extraction

```
from tensorflow.keras.applications import EfficientNetB0


base_model = EfficientNetB0(weights='imagenet', include_top=False, pooling='avg')

features = base_model.predict(image_data)   # generate embeddings
```

Snippet B3 – Decision Tree on Extracted Features

```
from sklearn.tree import DecisionTreeClassifier


dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train_features, y_train)

y_pred_dt = dt.predict(X_test_features)
```

Snippet B4 – Random Forest on Extracted Features

```
from sklearn.ensemble import RandomForestClassifier


rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_features, y_train)
y_pred_rf = rf.predict(X_test_features)
```

B.3 Execution Outputs

Confusion Matrices (Actual × Predicted)

Classes: **Gray leaf spot (GLS), Common rust, Northern Leaf Blight (NLB), Healthy**

Decision Tree (EfficientNet Features)

Actual \ Predicted	GLS	CR	NLB	Healthy
GLS	49	0	83	63
CR	0	120	0	0
NLB	83	0	10	6
Healthy	63	0	6	119

Random Forest (EfficientNet Features)

Actual \ Predicted	GLS	CR	NLB	Healthy
GLS	44	0	71	55
CR	0	120	0	0
NLB	71	0	11	16
Healthy	55	0	8	118

Convolutional Neural Network (CNN)

Actual \ Predicted	GLS	CR	NLB	Healthy
GLS	42	0	7	1
CR	0	120	0	0
NLB	7	0	91	1
Healthy	1	0	1	126

Classification Reports

Decision Tree

- Gray leaf spot: P=0.25, R=0.94, F1=0.40
- Common rust: P=1.00, R=1.00, F1=1.00
- Northern Leaf Blight: P=0.56, R=0.10, F1=0.17
- Healthy: P=0.94, R=0.63, F1=0.76
- Accuracy = **0.65**

Random Forest

- Gray leaf spot: P=0.26, R=0.85, F1=0.40
- Common rust: P=0.93, R=1.00, F1=0.96
- Northern Leaf Blight: P=0.55, R=0.11, F1=0.18
- Healthy: P=0.84, R=0.63, F1=0.72
- Accuracy = **0.64**

CNN

- Gray leaf spot: P=0.82, R=0.81, F1=0.82
- Common rust: P=1.00, R=1.00, F1=1.00
- Northern Leaf Blight: P=0.57, R=0.92, F1=0.70
- Healthy: P=0.98, R=0.67, F1=0.80
- Accuracy = **0.83**

Model Performance Summary

Model	Accuracy	Precision	Recall	F1-score	Inference Time (s)
Decision Tree	0.65	0.80	0.65	0.65	0.0018
Random Forest	0.64	0.74	0.64	0.63	0.1087
CNN	**0.83**	**0.88**	**0.83**	**0.83**	2.4668

B.4 Training Logs & Observations

- The **CNN** achieved the highest overall accuracy and F1-score, significantly outperforming Decision Tree and Random Forest on the extracted EfficientNet features.
- Both **Decision Tree** and **Random Forest** handled **Common rust** perfectly but struggled with **Northern Leaf Blight**, indicating feature overlap between disease classes.
- The CNN captured fine-grained texture differences, producing stronger recall for **Northern Leaf Blight** and balanced performance across all classes.

- There is a trade-off: while CNN is more accurate, it requires considerably longer inference time compared to Decision Tree.

B.5 Notes on Reproducibility

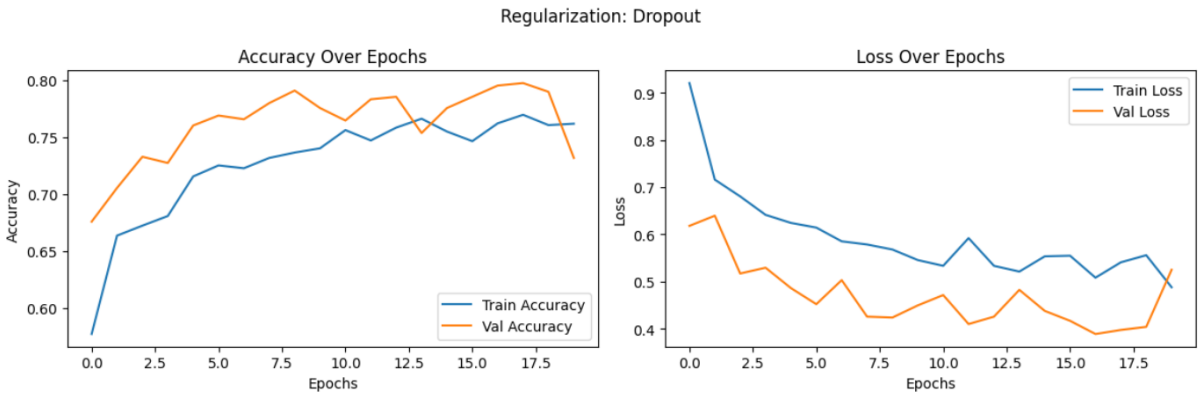
- **Frameworks**: TensorFlow/Keras, scikit-learn.
- **Data preprocessing**: Images resized to **224×224** and normalized.
- **Split ratio**: 80% training / 20% testing.
- **Random seed**: 42 for tree-based models.
- **Environment**: Jupyter Notebook with Python 3.11, TensorFlow 2.x, scikit-learn 1.4.

Appendix C – Model Performance Visualizations

This appendix presents the graphical representations of model training dynamics and evaluation results. While Appendix B contained the raw execution outputs (confusion matrices and classification reports), this section provides visualizations that make performance trends easier to interpret.

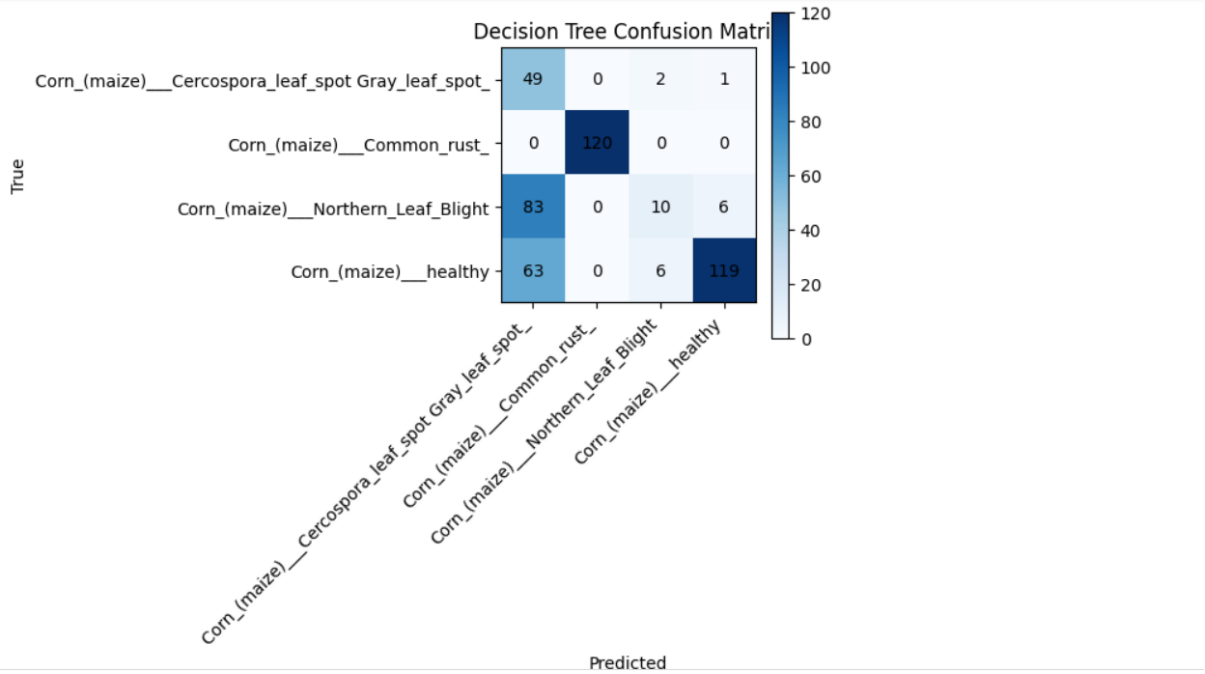
C.1 Training and Validation Accuracy/Loss

The training and validation curves of the CNN model are shown below. These curves illustrate how the model's performance evolved across epochs, highlighting convergence trends and potential overfitting or underfitting.



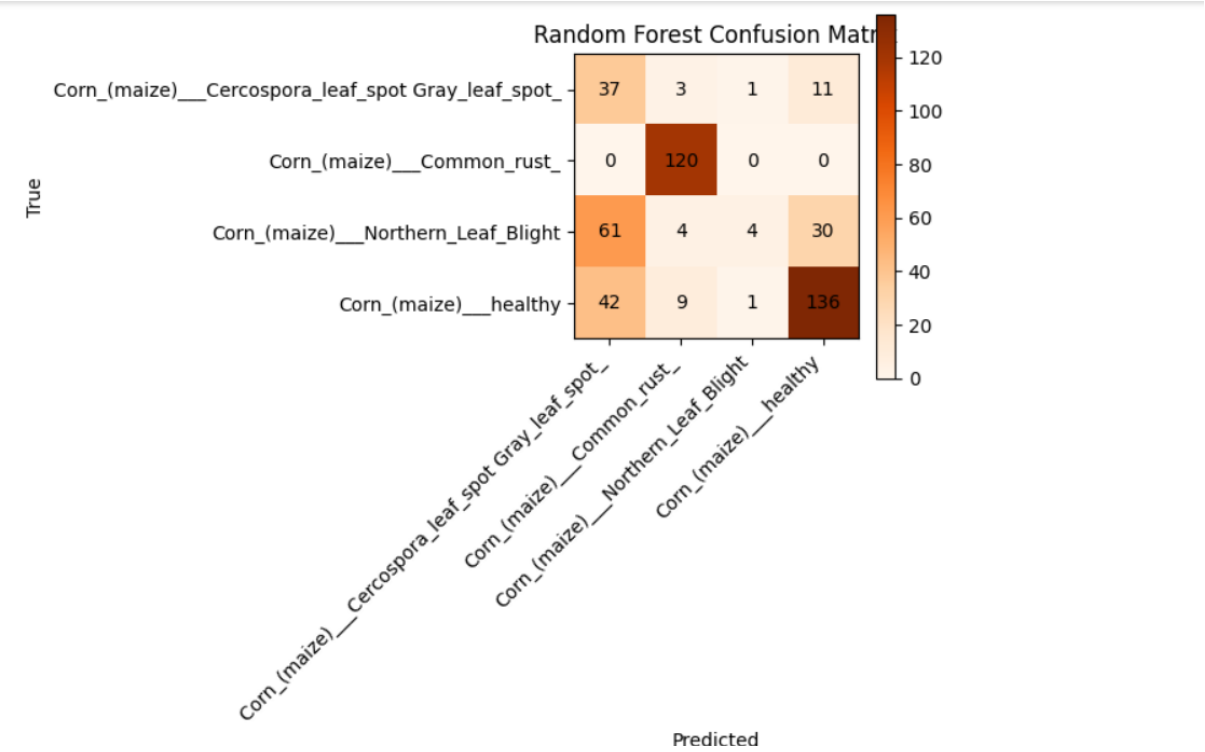
C.2 Confusion Matrix – Decision Tree

The Decision Tree classifier struggled with class separation, particularly for *Northern Leaf Blight*, as reflected in the confusion matrix. This indicates that simple tree-based methods were less effective in distinguishing fine-grained leaf textures.



C.3 Confusion Matrix – Random Forest

The Random Forest classifier performed slightly better than the Decision Tree but still fell short of CNN accuracy. The confusion matrix highlights similar misclassifications for *Northern Leaf Blight*, confirming the limitations of traditional feature-based models on this dataset.



C.4 Model Performance Comparison

To summarize, the figure below compares Accuracy, Precision, Recall, and F1-score across the three models (CNN, Decision Tree, Random Forest). This visualization clearly shows the superiority of the CNN in both predictive accuracy and balance across classes.



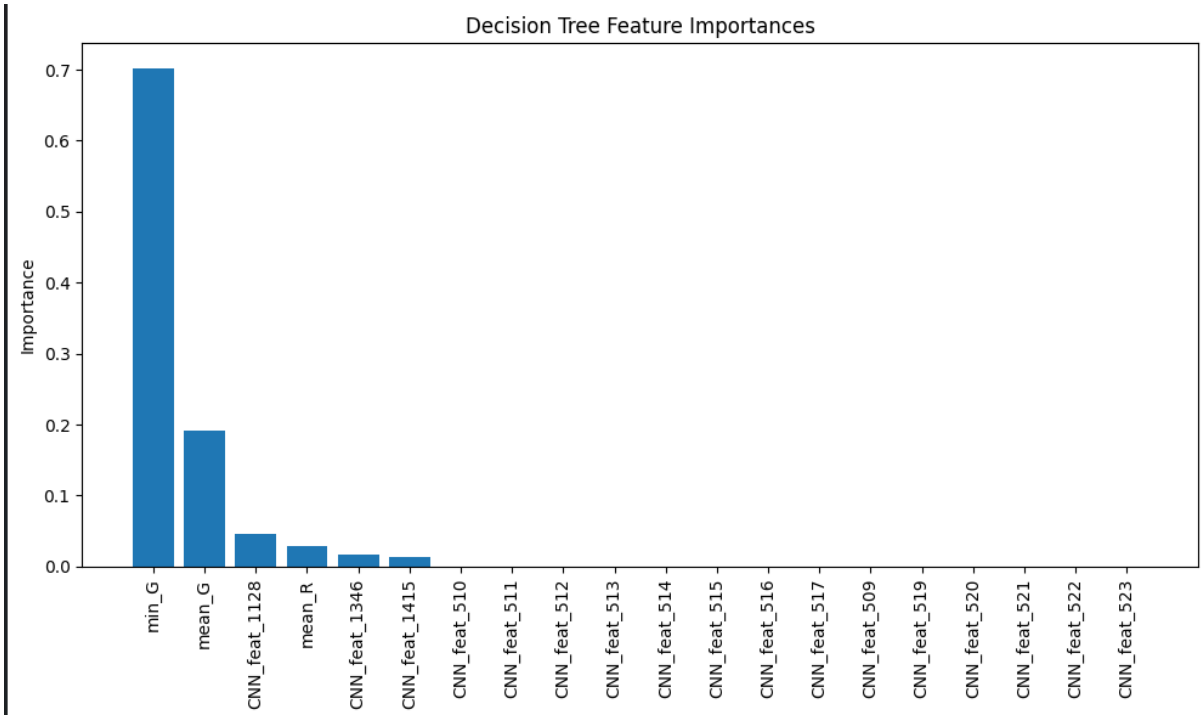
Appendix D – Explainability & Interpretability

This appendix presents model interpretability outputs for both **tree-based models** (Decision Tree, Random Forest) and the **CNN (Grad-CAM++)**. The purpose is to understand how models made predictions and where they encountered limitations.

D.1 Decision Tree & Random Forest – Feature Importance

Both the Decision Tree and Random Forest models were trained on features extracted using EfficientNet embeddings. Feature importance visualizations provide insights into which aspects of the input most strongly influenced predictions.

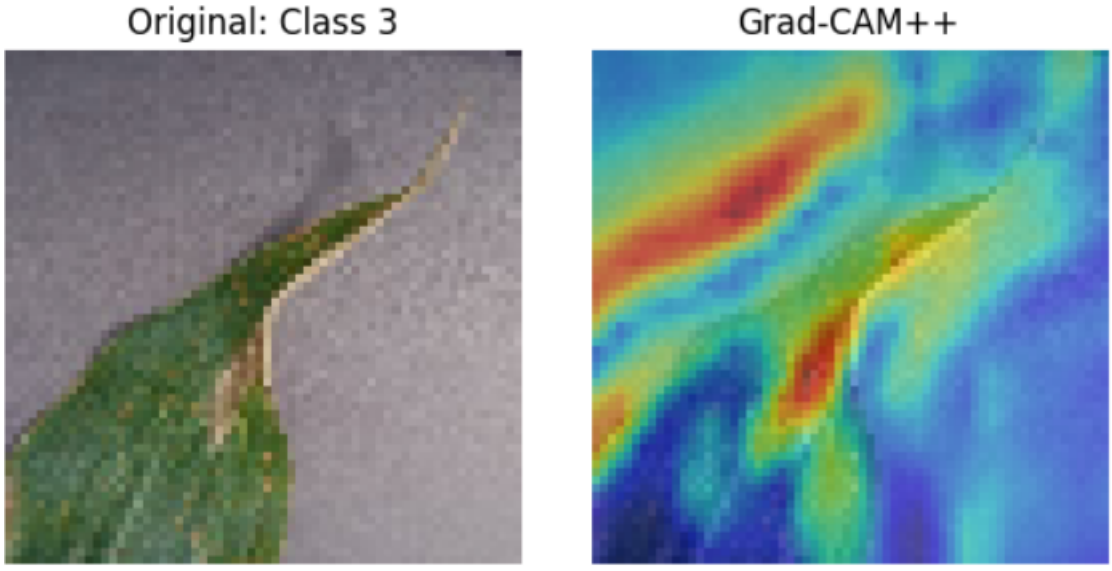
- The Decision Tree relied heavily on a small subset of features, leading to overfitting and poor generalization.



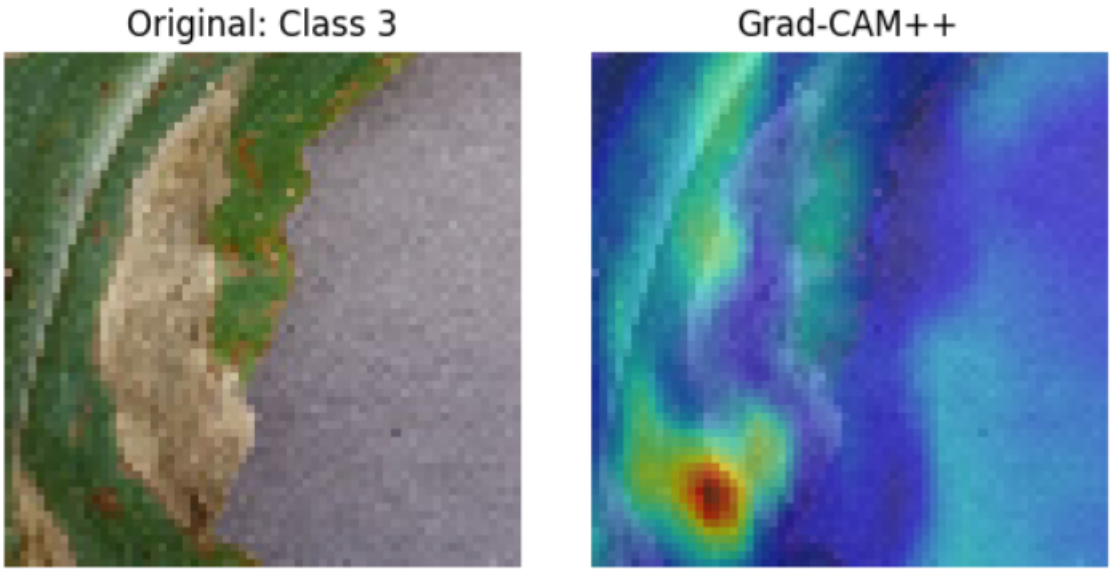
D.2 CNN – Grad-CAM++ Explanations

For the CNN, Grad-CAM++ heatmaps were applied to selected examples. Instead of multiple cases for each class, we present two representative scenarios: one correct prediction and one misclassification.

- **True Positive Example**
  A *Common Rust* leaf was correctly identified. Grad-CAM++ highlights lesion patches, showing the CNN focused on disease-relevant regions.



- **False Negative Example**
  A *Northern Leaf Blight* leaf was misclassified as Healthy. The heatmap shows attention on irrelevant areas, explaining the error.



Appendix E – Individual Contributions

Table E1 summarizes the responsibilities of each team member. The work was distributed to ensure coverage of preprocessing, classical ML approaches, deep learning models, transfer learning, explainability, and deployment considerations.

Table E1. Group Contributions

STUDENT NAME	CONTRIBUTION AREAS
MANIDEEP CHITTINENI	Development and training of the Convolutional Neural Network (CNN); Grad-CAM++ explainability (XAI) analysis; drafting Appendices A–C; contribution to model evaluation and interpretation.
SAKETH REDDY CHANDUPATLA	Data preprocessing pipeline (resizing, augmentation, normalization); quantization experiments for lightweight deployment; assistance with stratified dataset splitting and reproducibility.
YAMINI KONDRAGUNTA	Implementation of EfficientNetB3 transfer learning; feature extraction for Decision Tree and Random Forest; feature importance visualizations; contribution to Appendix D (tree-based explainability).
MAHABOOB AFROZ SHAIK	Evaluation of Decision Tree and Random Forest models; performance comparison tables/plots; literature review and report editing; final formatting and proofreading of the complete report.